

Parallel Image Blurring

Large-image box blur using OpenMP, Docker chunk workers, and MPI worker processes

Team Members and Work Distribution

B.No	Name	Work done
3	احمد رجب محمود	master.cpp
2	احمد باهي يوسف	mpi.cpp
9	زياد محمد هاشم	image_utils.cpp & worker.cpp
26	حمودي	blur.cpp

Item	Details
Repository	github.com/AAhmeddRRagabb/Parallel-Image-Blurring
Prepared for	HPC / Parallel Image Processing Project
Report focus	Code methodology, README validation, parallel chunking strategy, and experimental results
Input scale observed	10000 x 10000 RGB image

1. Executive Summary

The project implements a parallel image blurring system for very large RGB images. The main target is a square box blur, where each output pixel is computed from a square neighborhood around the same pixel location in the input image. The implementation compares three active execution methods: OpenMP, Docker-based distributed chunk processing, and MPI-based worker processing.

The code is designed around a practical high-performance computing challenge: a large image cannot be treated as a small local array when the filter is expensive. The image must be split into independent row chunks, assigned to workers, blurred in parallel, and merged back into one final image while preserving correctness at chunk borders.

Most important design point: the implementation uses row-wise chunking with halo rows. Each worker receives its assigned output rows plus additional neighboring rows above and below the chunk. These extra rows allow the blur kernel to compute boundary pixels correctly without requiring continuous communication during computation.

The measured results show that OpenMP is the fastest implementation in the committed experiment data. MPI is slower than OpenMP but much faster than the Docker-based method. Docker has the highest overhead because each chunk is processed through container creation, mounting, logging, waiting, and cleanup.

2. Problem Definition

The problem is to apply a blur operation to a very large image. The repository includes an input image of size 10000 x 10000 pixels, which creates a heavy workload because each pixel requires repeated access to neighboring pixels. With a 25 x 25 filter, every output pixel may inspect up to 625 neighboring locations. For a 100 million-pixel image, this becomes a substantial computation.

A naive blur over a large image is computationally expensive. Parallelization improves performance by dividing the image into independent row bands. Each worker computes one band, then the master process merges the bands in their original order. Blur computation depends on neighboring rows, so pixels near chunk edges require halo rows.

Core requirements

- Load a large color image.
- Apply a square blur filter with configurable filter sizes.
- Support zero, mirror, and none padding modes.
- Split the large image into row chunks and distribute the chunks to workers.
- Compare OpenMP, Docker, and MPI execution time.
- Save output images and timing logs for analysis.

3. Repository Structure and Main Files

File / Folder	Role in the project
run.sh	Main experiment runner. Compiles the C++ code, prepares the Docker image, clears/creates the log file, and runs experiments for selected filter sizes and

	padding modes.
master.cpp	Main coordinator. Loads the image, runs OpenMP blur, Docker chunk blur, and MPI blur, saves output images, and logs timings.
worker.cpp	Docker worker program. Reads one chunk and metadata file, applies blur to the chunk, and writes the blurred output chunk.
blur.cpp / blur.h	Core blur logic: serial blur, OpenMP blur, Docker chunk blur, mirror indexing, and pixel access with padding rules.
blur_mpi.cpp / blur_mpi.h	MPI master-worker distribution. Sends chunk metadata and pixel data to worker ranks and receives processed chunks.
image_utils.cpp / image_utils.h	Image loading, saving, and chunk extraction with halo rows.
Dockerfile	Builds an Ubuntu-based OpenCV worker image and compiles the Docker worker executable.
input_output/	Contains the large input image and committed comparison assets such as charts and CSV timing summaries.

4. README Validation Against the Code

The README explains the project as a comparison between Serial, OpenMP, Docker, and MPI strategies. The code base supports these concepts, but the currently committed runner and master program differ from some README statements.

README statement	Code validation	Status
The project compares Serial, OpenMP, Docker, and MPI.	The code contains serial blur logic, but the serial execution block in master.cpp is commented out. The active run compares OpenMP, Docker, and MPI.	Partially consistent
run.sh runs 27 experiments: 3 padding modes x 3 filter sizes x 3 worker counts.	The committed run.sh uses WORKERS_LIST=(4), FILTER_SIZES=(15 25), and 3 padding modes. This produces 6 runs, not 27.	Inconsistent
Filter sizes include 15, 31, and 51.	The committed runner uses 15 and 25.	Inconsistent
Default input path is input_output/input.jpg.	The committed runner defaults to input_output/input.png.	Inconsistent
The output includes a serial image.	master.cpp still prints/saves paths mentioning serial output, but serial computation is disabled.	Inconsistent
Docker and MPI use chunking.	The code clearly implements row chunking with halo rows for both Docker and MPI.	Consistent
Supported padding modes are zero, mirror, and none.	The implementation contains all three padding modes in the	Consistent

	shared blur logic.	
--	--------------------	--

Validation conclusion: the README is directionally correct, but it should be updated to match the current experiment configuration. The strongest mismatches are the number of runs, active filter sizes, default input extension, and inactive serial benchmark.

5. Detailed Methodology Found in the Code

5.1 Blur operation

The project applies a square box blur. For each output pixel, the program scans a square neighborhood around the corresponding input pixel. The filter radius is calculated as half of the filter size. For example, a 25 x 25 filter has radius 12. For valid neighboring pixels, the RGB channels are accumulated and divided by the number of contributing pixels. The behavior at image borders depends on the selected padding mode.

5.2 OpenMP method

The OpenMP implementation parallelizes the outer row loop of the blur operation. This is the simplest parallel strategy in the project because all worker threads share the same memory. No explicit chunk files, containers, or MPI messages are needed.

5.3 Chunking with halo rows

Both the Docker and MPI implementations use row-wise chunking. The master process divides the input image into horizontal row bands. For each band, it also extracts halo rows above and below the assigned output rows. The number of halo rows is equal to the blur radius. After processing, the blurred chunk output contains only the assigned output rows, not the halo rows.

5.4 Docker-based chunk workers

In the Docker workflow, master.cpp writes each chunk and metadata file to the output directory. Then it starts one Docker container per chunk. Each container runs the compiled worker program from the Docker image, reads its assigned chunk, applies the same blur logic, writes the output chunk, and exits.

5.5 MPI-based workers

In the MPI workflow, rank 0 acts as the master and ranks 1..N act as workers. The master sends metadata and raw chunk pixel data through MPI messages. Worker ranks receive one chunk, blur it, and send the result buffer back. Extra idle ranks are stopped using a completion message.

Padding modes

Padding mode	Implementation behavior
Zero	Out-of-image neighbors are treated as black pixels. These zero values still participate in the average.
Mirror	Out-of-image coordinates are reflected back into the valid image range using mirror indexing.
None	Out-of-image neighbors are ignored. Only valid pixels contribute to the average.

6. Experiment Setup and Timing Data

The committed timing CSV files show six experiment runs. All runs use four workers. The tested filter sizes are 15 and 25. The tested padding modes are zero, mirror, and none.

Run	Workers	Filter	Padding	OpenMP (s)	MPI (s)	Docker (s)
1	4	15	zero	13.6528	35.1558	53.3600
2	4	25	zero	42.7746	87.5965	121.6540
3	4	15	mirror	15.7778	34.9196	53.1821
4	4	25	mirror	43.5118	87.3795	119.5980
5	4	15	none	14.9510	37.6365	55.2682
6	4	25	none	41.7856	89.0026	122.6400

7. Results and Discussion

7.1 Visual output example - 25 x 25 blur

The result images below use the compressed images supplied for this reconstructed report. The first image is the original input and the second image is the blurred output. The blurred result represents the visual effect of applying the 25 x 25 box blur. Because OpenMP, MPI, and Docker call the same blur logic, the visual output should be equivalent for the same filter size and padding mode; their main difference is execution time.

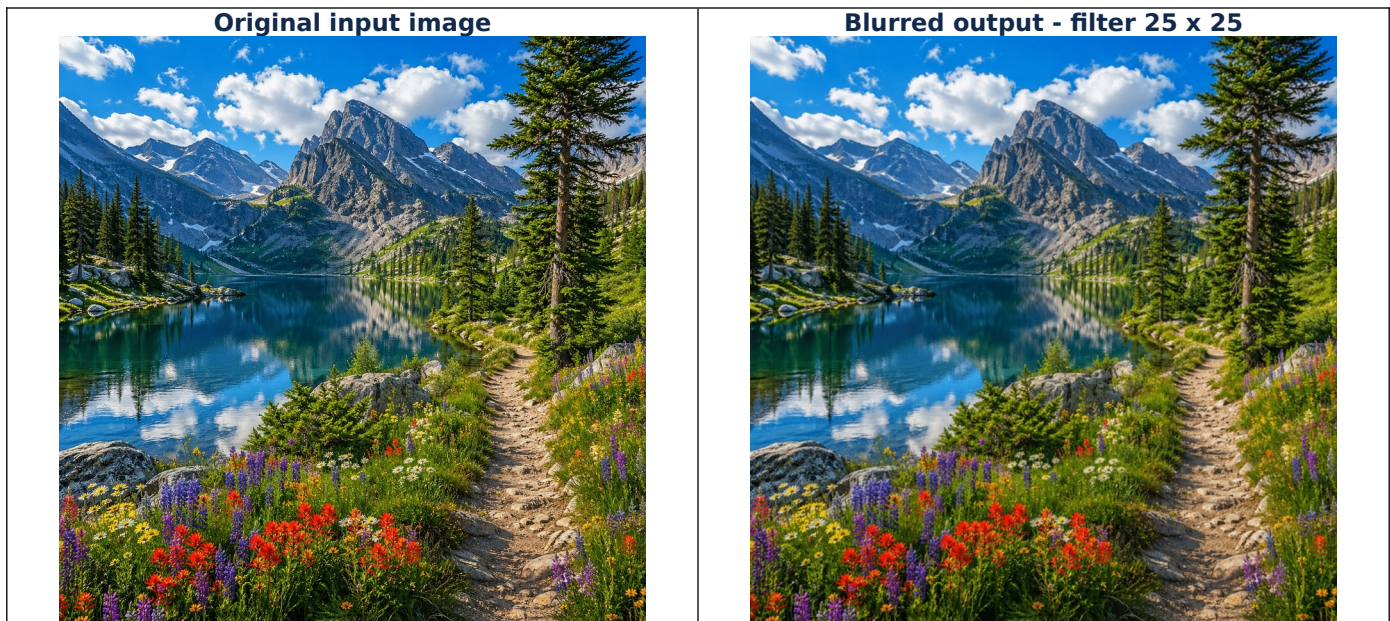


Figure 1. Original input image and 25 x 25 blurred output sample using the supplied compressed images.

7.2 Time comparison by filter size

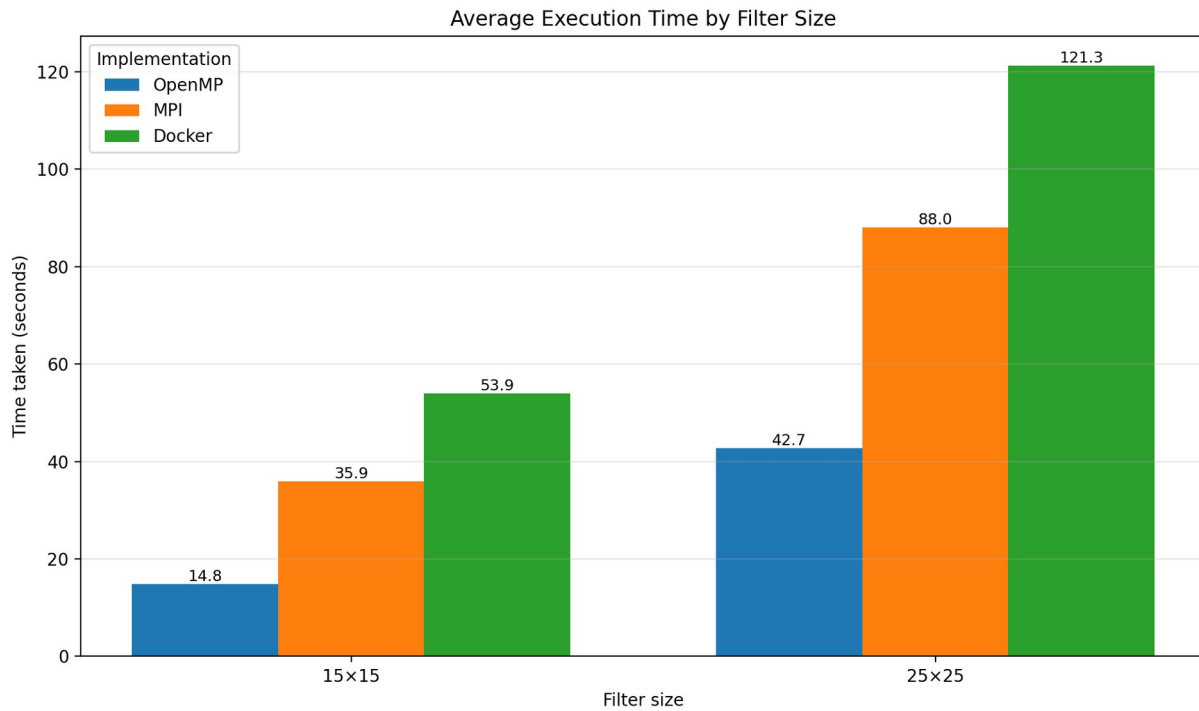


Figure 2. Average execution time grouped by filter size.

The filter-size comparison shows a strong increase in execution time when moving from 15 x 15 to 25 x 25. This is expected because the computational cost of the blur grows with the filter area. A 15 x 15 filter checks up to 225 neighbor positions per pixel, while a 25 x 25 filter checks up to 625 positions per pixel.

Filter size	OpenMP avg (s)	MPI avg (s)	Docker avg (s)
15	14.7939	35.9040	53.9368
25	42.6907	87.9929	121.2973

7.3 Time comparison by padding mode

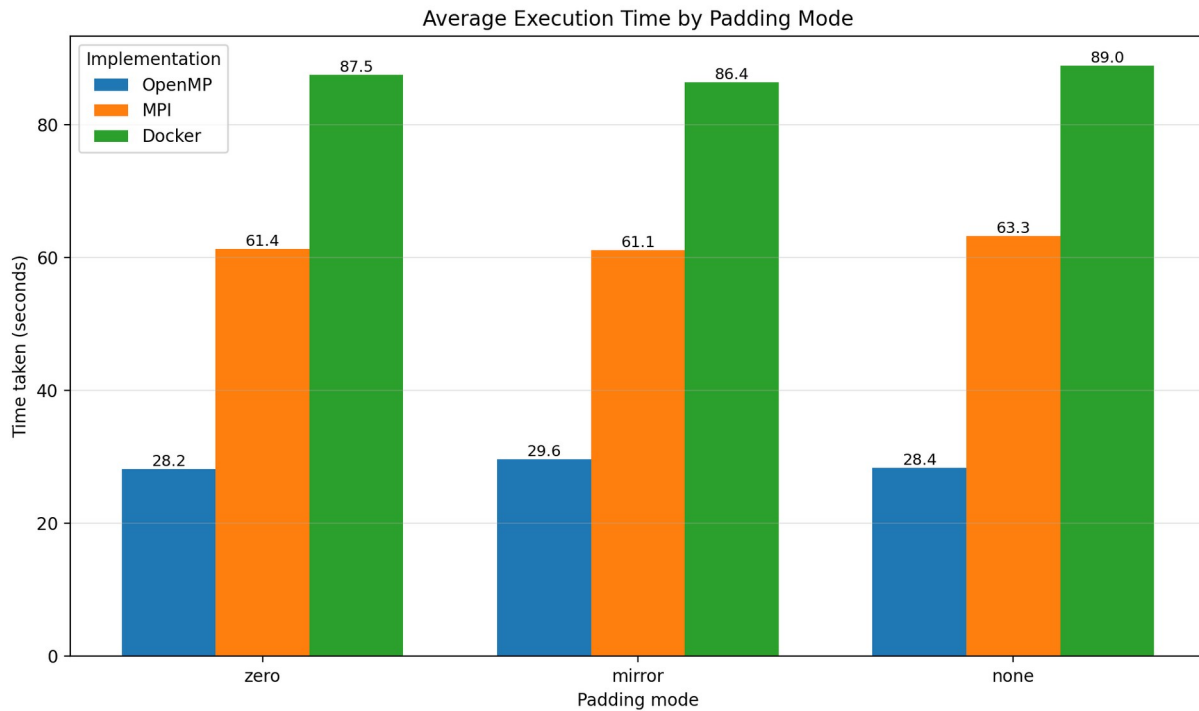


Figure 3. Average execution time grouped by padding mode.

The padding-mode comparison shows relatively small differences compared with the effect of filter size. This indicates that the dominant cost is the number of neighbor operations per pixel, while the padding branch logic is a secondary factor.

Padding mode	OpenMP avg (s)	MPI avg (s)	Docker avg (s)
zero	28.2137	61.3762	87.5070
mirror	29.6448	61.1496	86.3901
none	28.3683	63.3196	88.9541

7.4 Performance interpretation

- OpenMP is fastest because it uses shared-memory threads and avoids external orchestration overhead.
- MPI is second because it distributes work through message passing. It has communication overhead, but less overhead than Docker containers.
- Docker is slowest because each worker requires file-based chunk exchange, container launch, container wait, log capture, and cleanup.
- Filter size dominates runtime because increasing the filter from 15 to 25 increases the maximum neighborhood operations per pixel from 225 to 625.
- Padding mode has limited timing impact because only border neighborhoods require special behavior; most pixels are inside the image.

8. Limitations and Recommended Improvements

8.1 Current limitations

- The README does not fully match the committed experiment runner.
- The serial benchmark is present in code but inactive in the current master workflow.
- The current Docker design has high orchestration overhead and is not ideal for pure performance on a single machine.
- The blur kernel is a direct neighborhood loop. For large box filters, this is expensive.
- The committed repository listing did not expose full result images under an `input_output/results` directory.

8.2 Recommended improvements

- Update the README to match the active filter sizes, worker list, default input filename, and number of runs.
- Either re-enable the serial benchmark or remove serial output messages from the active run.
- Add a correctness check that compares OpenMP, Docker, and MPI output images pixel-by-pixel for the same configuration.
- Use an optimized box blur method based on integral images or separable running sums to reduce the cost of large filters.
- For Docker experiments, consider persistent worker containers instead of launching a new container per chunk.
- Commit a small sample result folder so readers can verify output structure without downloading very large generated images.

9. Conclusion

The project is a strong practical demonstration of parallel image processing on a large image. It implements a clear shared blur core and compares three active execution models: OpenMP, Docker chunk workers, and MPI worker processes. The central technical idea is correct and important: row-wise chunking with halo rows enables independent worker computation while preserving blur correctness at chunk boundaries.

The experimental results support the expected performance order for this environment: OpenMP is fastest, MPI is slower due to communication overhead, and Docker is slowest due to container and file orchestration overhead. The results also show that filter size is the main driver of runtime, while padding mode has a smaller effect.

With README cleanup, output validation, and possible blur-kernel optimization, this repository can serve as a polished and convincing HPC project showing how large data can be distributed across workers using both container-based and MPI-based approaches.